

AIRPORT RUNWAY MANAGEMENT SIMULATOR

Software Development Project Report

Project Type: Java Desktop Application

Technology: Java Swing, Object-Oriented Programming

Development Period: 10 Weeks

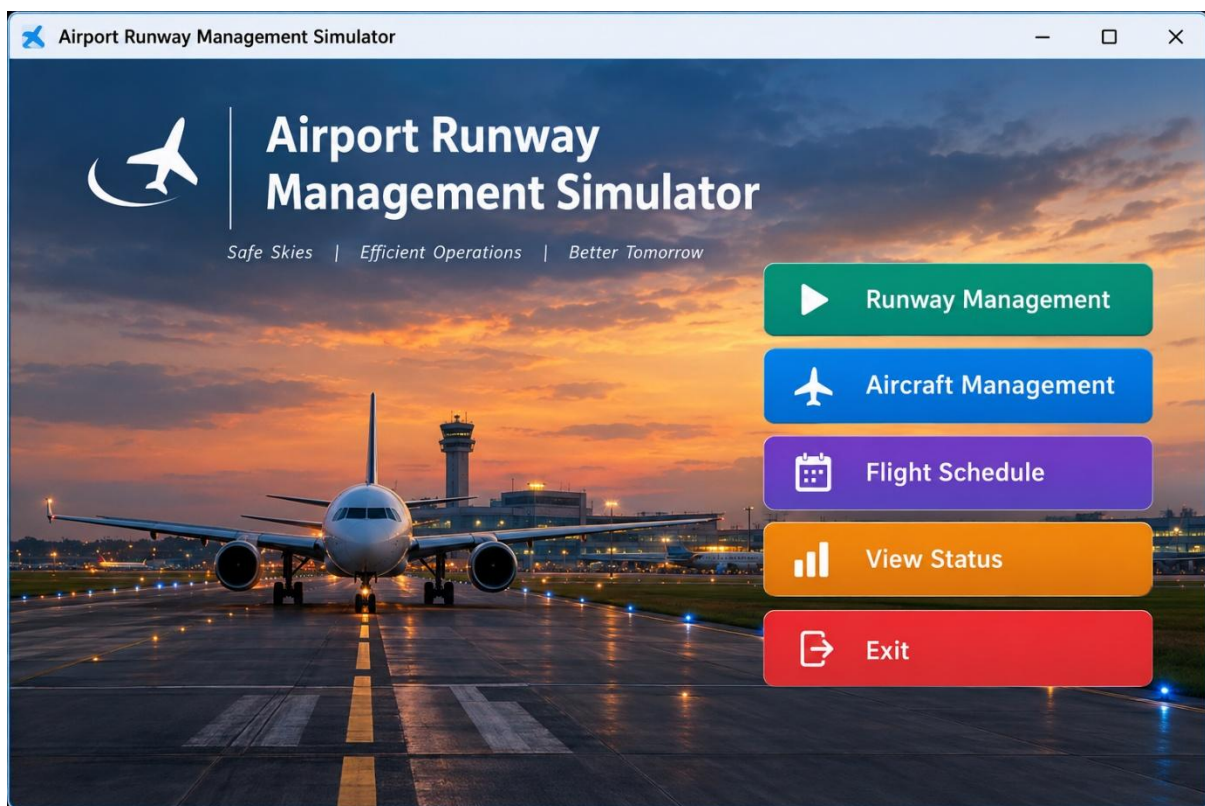
Final Project Report

1. Introduction

The Airport Runway Management Simulator is a Java-based interactive desktop application developed as a software development project. The main purpose of the software is to provide a simulated environment for managing airport runways, aircraft information, landing and takeoff requests, runway allocation, emergency aircraft handling, and operational status.

The application uses Java Swing to create the graphical user interface and follows Object-Oriented Programming concepts to organize the different components of the system. The software provides a main menu, runway management, aircraft management, runway allocation, emergency handling, status monitoring, and reporting facilities.

The project was developed incrementally. Different features were implemented, tested, and improved step by step throughout the development period.



2. Project Objectives

- To develop an interactive airport runway management application using Java.
- To create a simple and user-friendly graphical user interface.
- To manage runway information and runway availability.
- To store and manage aircraft information.
- To register landing and takeoff requests.
- To allocate available runways to aircraft requests.
- To process landing and takeoff requests in an organized manner.
- To provide priority handling for emergency aircraft.
- To display runway and aircraft status.
- To generate operation reports and summary information.
- To apply Object-Oriented Programming concepts.
- To use suitable data structures such as ArrayList, Queue, and Priority Queue.
- To use GitHub for source-code management and project development.

3. Project Scope

The project focuses on developing a standalone desktop-based Airport Runway Management Simulator using Java and Java Swing. The user can start the application from the main menu and access different airport management functions.

The system covers runway management, aircraft registration and records, landing and takeoff request processing, automatic runway allocation, emergency priority handling, status monitoring, and operation reporting. The project is designed as a simulation and does not directly control real airport equipment or aircraft.

The project can be extended in future with database integration, real-time flight data, advanced scheduling, authentication, and more realistic airport simulation features.

4. Development Methodology

The project was developed using the Incremental Development Model. In this model, the complete system is divided into smaller increments, and each increment adds a specific functionality to the software. The project was planned and developed over ten weeks.

Week	Development Work
Week 1	Project Planning and Requirement Analysis
Week 2	Software Design
Week 3	User Interface Development
Week 4	Aircraft Management System
Week 5	Runway Allocation System
Week 6	Integration and Improvement of Core Module
Week 7	Emergency Handling System
Week 8	Status Monitoring and Reporting
Week 9	Software Finalization and UI Improvement
Week 10	Testing and Final Release

5. Software and Hardware Requirements

5.1 Software Requirements

- Java Development Kit (JDK)
- NetBeans IDE
- Java Swing
- GitHub
- Windows Operating System

5.2 Hardware Requirements

The application does not require high-end hardware. A normal computer or laptop capable of running Java and NetBeans is sufficient.

- Computer or Laptop
- Keyboard and Mouse
- Sufficient RAM
- Sufficient storage space

6. System Features

6.1 Main Menu

The Main Menu is the starting interface of the application. It provides navigation to the major modules of the simulator.



Figure 1: Main Menu

The Runway Management option opens runway-related functions such as adding, removing, viewing, and assigning runways.

6.2 Runway Management

The Runway Management module is responsible for maintaining runway information and availability. The user can add runways, remove runways, view available runways, and assign a runway to an aircraft request.

The Runway Management interface features a sidebar with five buttons: 'Add Runway', 'Remove Runway', 'View Runways', 'Assign Runway', and 'Back to Main Menu'. The main area contains a table with the following data:

Runway ID	Status	Length (m)	Type
RWY 01	Available	3000	Departures
RWY 19	Occupied	3000	Arrivals
RWY 27	Available	2800	Departures
RWY 09	Available	2800	Arrivals

Figure 2: Runway Management

6.3 Aircraft Management System

The Aircraft Management System stores and manages aircraft information. It provides a structured way to register aircraft and maintain their current operational status.

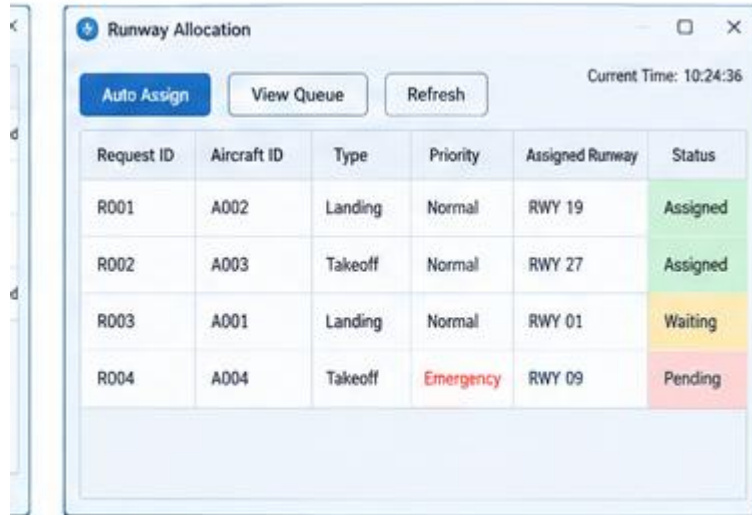
The Aircraft Management interface includes input fields for 'Aircraft ID', 'Model', 'Airline', 'Type', 'Flight Number', and 'Status'. Below these fields are 'Register', 'Update', and 'Clear' buttons. The main area displays a table with the following data:

Aircraft ID	Model	Airline	Type	Flight No	Status
A001	B737	Biman	Passenger	BG101	Schedule
A002	A320	US-Bangla	Passenger	BS201	Landing
A003	B777	Emirates	Passenger	EKS65	Takeoff
A004	B737	Biman	Passenger	BG102	Schedule

Figure 3: Aircraft Management

6.4 Runway Allocation System

The Runway Allocation System assigns available runways to landing and takeoff requests. The system checks runway availability and processes requests according to the management logic implemented in the application.



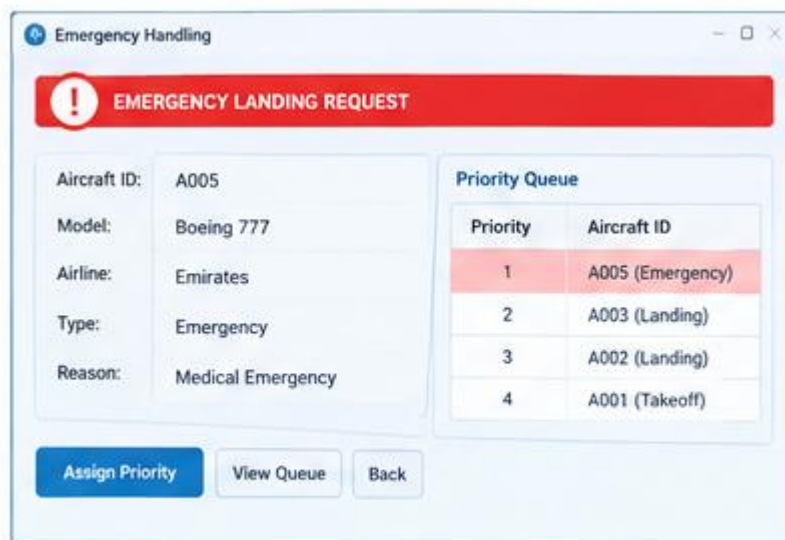
The screenshot shows a window titled "Runway Allocation" with a current time of 10:24:36. It contains three buttons: "Auto Assign", "View Queue", and "Refresh". Below these is a table with the following data:

Request ID	Aircraft ID	Type	Priority	Assigned Runway	Status
R001	A002	Landing	Normal	RWY 19	Assigned
R002	A003	Takeoff	Normal	RWY 27	Assigned
R003	A001	Landing	Normal	RWY 01	Waiting
R004	A004	Takeoff	Emergency	RWY 09	Pending

Figure 4: Runway Allocation

6.5 Emergency Handling System

The Emergency Handling System provides priority treatment for emergency aircraft. Emergency landing requests are given higher priority so that the simulator can demonstrate priority-based airport traffic management.



The screenshot shows a window titled "Emergency Handling" with a red banner at the top that reads "EMERGENCY LANDING REQUEST". Below the banner, there is a form with the following details:

- Aircraft ID: A005
- Model: Boeing 777
- Airline: Emirates
- Type: Emergency
- Reason: Medical Emergency

To the right of the form is a "Priority Queue" table:

Priority	Aircraft ID
1	A005 (Emergency)
2	A003 (Landing)
3	A002 (Landing)
4	A001 (Takeoff)

At the bottom of the window are three buttons: "Assign Priority", "View Queue", and "Back".

Figure 5: Emergency Handling

6.6 Status Monitoring and Reporting

The Status Monitoring and Reporting module displays current runway status, aircraft information, and operational summaries. It provides a summary view that helps the user understand the current state of the airport simulation.

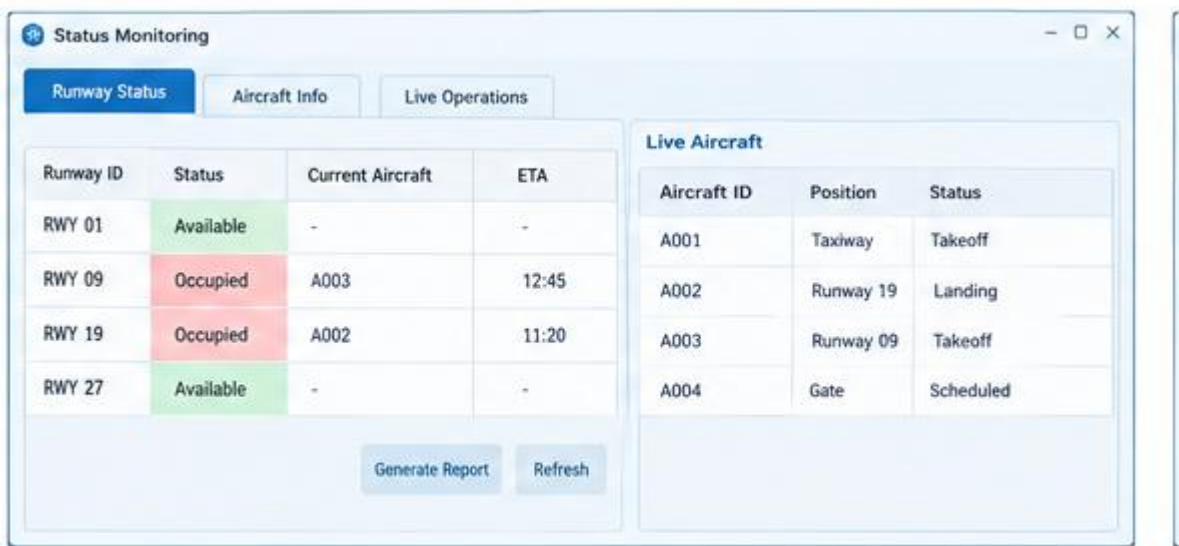


Figure 6: Status Monitoring

7. System Architecture

The system is divided into several classes and functional modules. Each module has a specific responsibility, making the software easier to understand, test, and maintain.

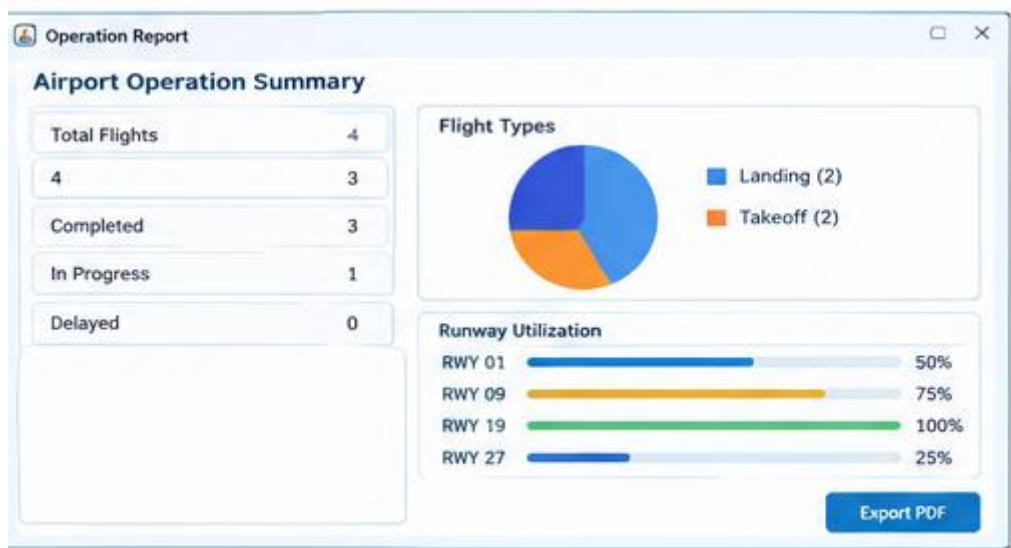


Figure 7: Operation Report / Summary

8. Project Structure

The Java source files are organized into logical classes and modules. The project structure separates user-interface components from airport-management logic.

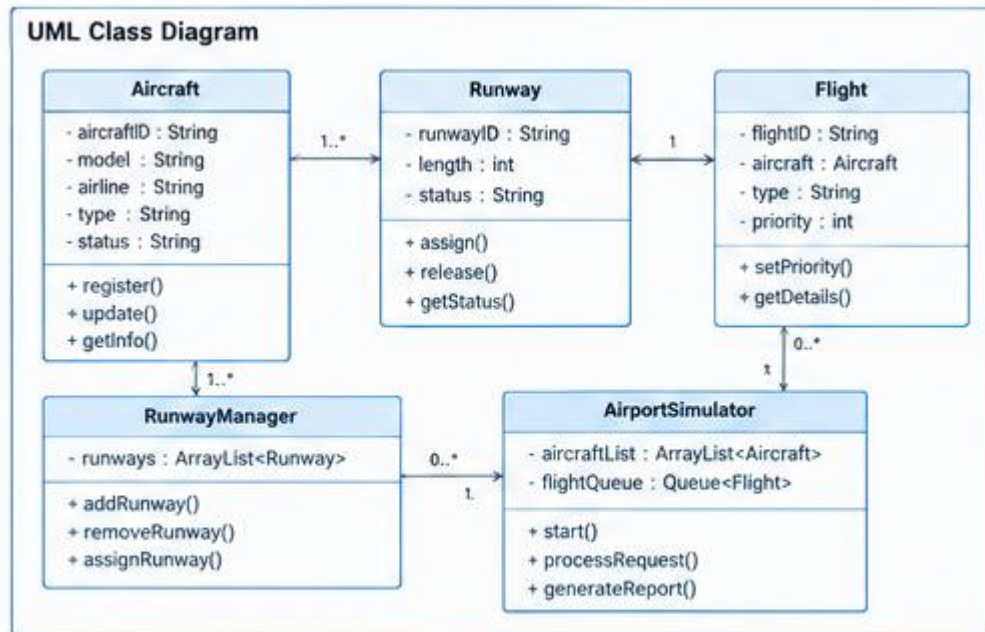


Figure 8: Class Diagram

9. Object-Oriented Programming Concepts Used

9.1 Class and Object

The application is divided into classes representing different system responsibilities, such as runway, aircraft, request, and user-interface components. Objects are created from these classes to perform different operations.

9.2 Encapsulation

Important data such as aircraft information, runway status, and request information can be kept inside their respective classes and accessed through appropriate methods.

9.3 Inheritance

The GUI classes can inherit from Java Swing classes such as `JFrame`. For example, a menu or management window can be created by extending `JFrame`.

9.4 Modularity

Different responsibilities are divided among different classes and modules. This makes the project easier to develop, test, debug, and maintain.

10. Data Structures Used

10.1 ArrayList

ArrayList can be used to store and manage aircraft and runway records dynamically. It allows records to be added, removed, searched, and updated.

10.2 Queue

A Queue is suitable for managing normal landing and takeoff requests in an ordered manner. Requests can be processed according to their arrival order.

10.3 Priority Queue

A Priority Queue is used for emergency handling concepts. Emergency aircraft can receive higher priority than normal requests.

These data structures support the main airport-management operations and demonstrate practical application of Java Collections Framework.

11. System Rules

- A runway must be available before it can be assigned.
- An occupied runway cannot be assigned to another aircraft at the same time.
- Aircraft requests must contain valid information.
- Landing and takeoff requests are processed through the request-management system.
- Emergency aircraft receive priority according to the emergency-handling logic.
- Runway status is updated after an assignment or operation.
- Aircraft status is updated according to its current operation.
- Invalid operations should not change valid system data.
- The user can return to the main menu from individual modules.

12. System Workflow

The overall workflow of the Airport Runway Management Simulator can be represented as follows:

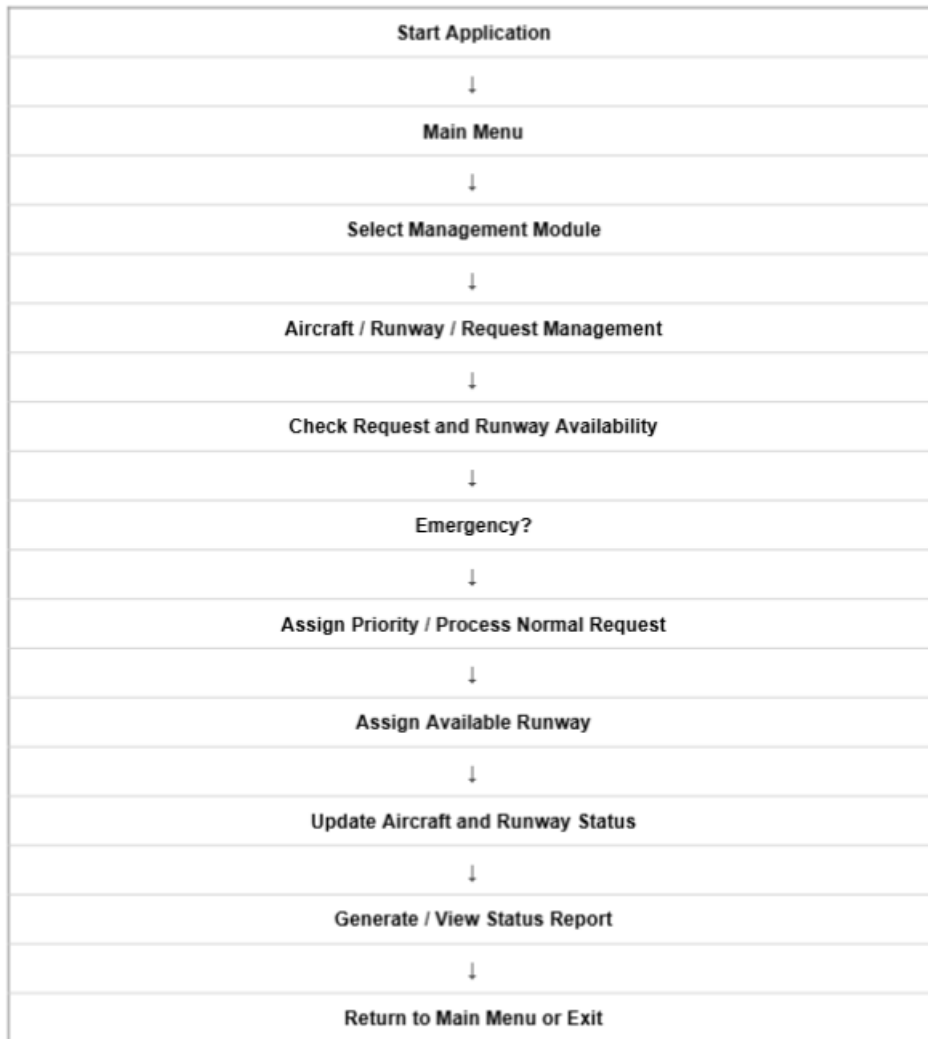


Figure: Overall Workflow of Airport Runway Management Simulator

13. Testing

Testing was performed to verify the functionality of the software. The main test cases are summarized below.

Test Case	Expected Result
Start Application	Main Menu open
Open Runway Management	Runway Management Screen Open
Add Runway	New runway record is added
Remove Runway	Selected runway is removed
View Runway	Runway information is displayed
Resister Aircraft	Aircraft record is stored
Landing Request	Landing request is processed
Takeoff Request	Takeoff request is processed
Assign Runway	Available Runway is assigned
Occupied Runway	Assignment is prevented
Emergency Request	Emergency request receives priority
View Status	Current runway and aircraft status appears
Generate Report	Operation summary is displayed
Exit	Application closes

14. Error Handling and Validation

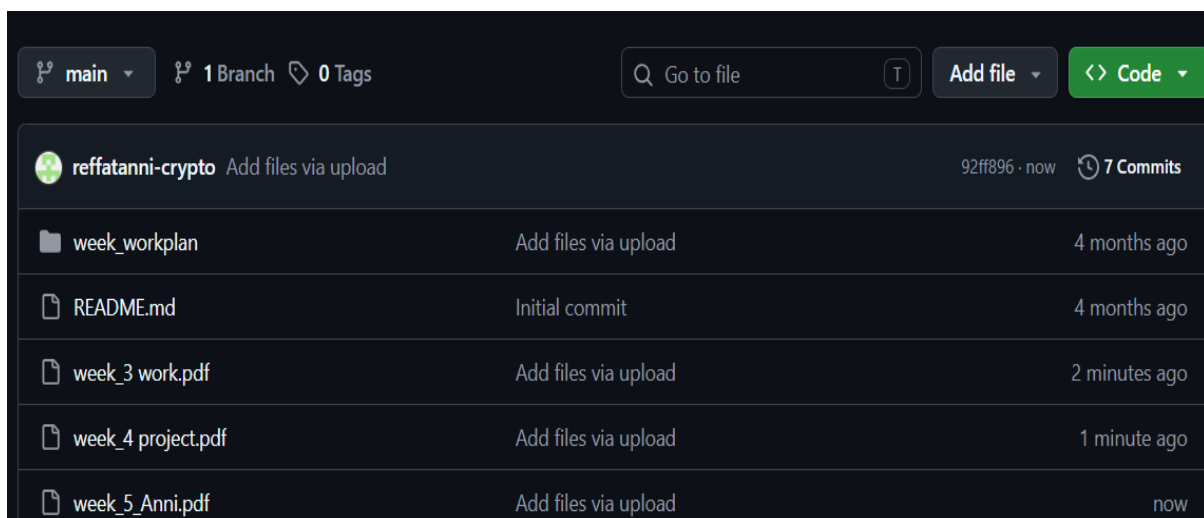
The system validates important user actions before changing the airport state. Validation helps prevent invalid

runway assignment, incorrect records, and inconsistent operational status.

- Check whether the selected runway exists.
- Check whether a runway is currently available.
- Validate aircraft information before storing it.
- Validate landing and takeoff requests.
- Prevent assignment when no suitable runway is available.
- Handle emergency requests through priority logic.
- Keep system status consistent after each operation.

15. GitHub Version Control

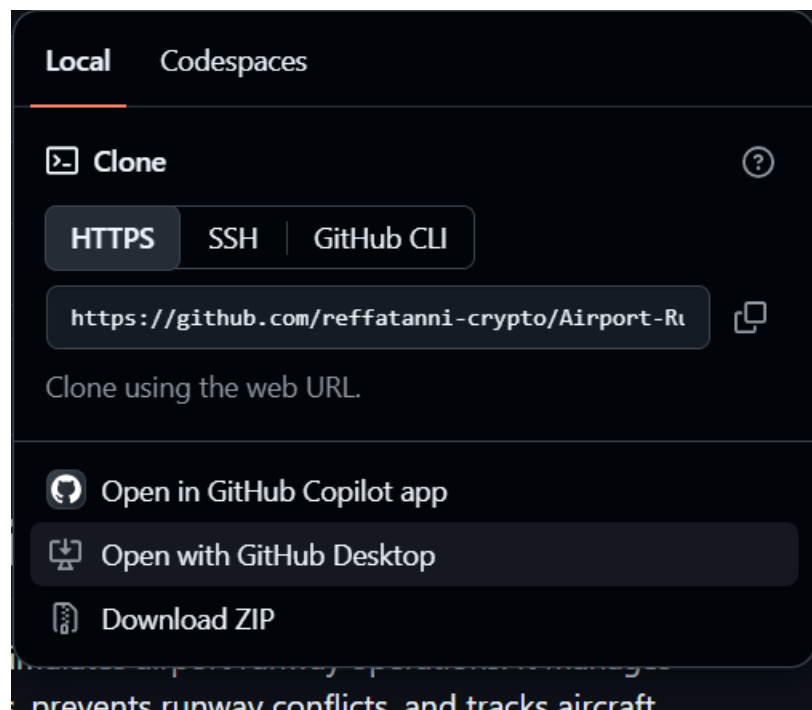
GitHub was used to store and manage the source code and project documentation. During development, different modules and project files were uploaded according to the development schedule.



16. Downloading the Project from GitHub

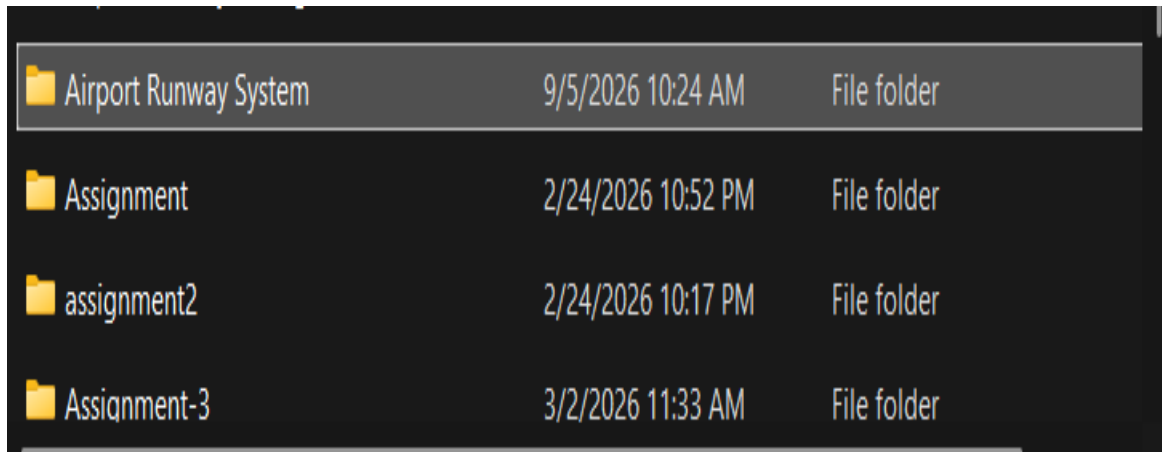
Another user can download and use the Airport Runway Management Simulator from the project's GitHub repository.

- Step 1: Open the GitHub repository.
- Step 2: Click the Code button.
- Step 3: Select Download ZIP.
- Step 4: Save the ZIP file to the computer.
- Step 5: Extract the downloaded ZIP file



17. Opening the Project in IntelliJ Idea

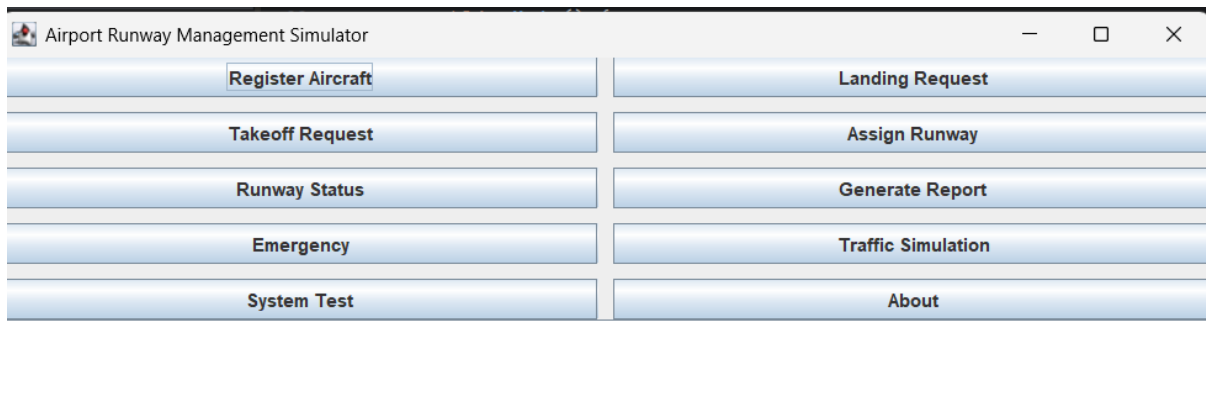
- Open IntelliJ idea.
- Select File → Open Project.
- Select the extracted Airport Runway Management Simulator project folder.
- Click Open Project.
- Wait for the project to load.
- Check the source packages and Java files

A screenshot of a file explorer window with a dark theme. It displays a list of four folders. Each folder entry consists of a yellow folder icon, the folder name, the last modified date and time, and the file type 'File folder'. The first folder, 'Airport Runway System', is highlighted with a light gray background.

 Airport Runway System	9/5/2026 10:24 AM	File folder
 Assignment	2/24/2026 10:52 PM	File folder
 assignment2	2/24/2026 10:17 PM	File folder
 Assignment-3	3/2/2026 11:33 AM	File folder

18. Running the Project

- Run the project from IntelliJ Idea.
- The Main Menu will appear.
- Select the required management module.
- Register or view aircraft information.
- Manage runways and process requests.
- Assign an available runway.
- Test emergency priority handling.
- View status and operation reports.
- Exit the application when finished



19. How Another User Can Use the Software



20. Challenges Faced

- Designing a suitable airport runway-management workflow.
- Creating the graphical user interface.
- Managing runway records and availability.
- Storing and updating aircraft information.
- Processing landing and takeoff requests.
- Implementing automatic runway allocation.
- Handling runway occupancy correctly.
- Implementing emergency priority handling.
- Connecting GUI components with management logic.
- Updating aircraft and runway status.
- Generating operation summaries and reports.
- Testing and debugging the complete application

These challenges provided practical experience in Java programming, Object-Oriented Programming, GUI development, data structures, event-driven programming, and software development.

21. Result

The Airport Runway Management Simulator was successfully developed as an interactive Java Swing application. The final application provides a graphical environment for managing airport runways and aircraft operations in a simulated setting.

The system supports runway management, aircraft management, landing and takeoff request processing, runway allocation, emergency priority handling, status monitoring, and reporting. The project also demonstrates practical use of Java, Object-Oriented Programming, Java Swing, event handling, ArrayList, Queue, Priority Queue, and GitHub version control.

22. Future Improvements

- Database integration for permanent aircraft and runway records.
- Real-time flight information integration.
- Multiple airport support.
- Advanced flight scheduling.
- More realistic runway occupancy simulation.
- User login and role-based access.
- Detailed graphical airport map.
- Sound and visual emergency alerts.
- Advanced reporting and statistics.
- Improved UI design and animations.
- Dark mode.
- Cloud-based data storage.
- More advanced conflict-detection and scheduling algorithms

23. Conclusion

The Airport Runway Management Simulator project provided practical experience in the complete software development process. The project started with requirement analysis and design and was gradually developed through GUI development, aircraft management, runway management, request processing, runway allocation, emergency handling, status monitoring, testing, and final documentation.

The project demonstrates the practical application of Java, Object-Oriented Programming, Java Swing, event handling, data structures, priority-based processing, validation, and GitHub version control. The final application provides an interactive environment where users can manage simulated airport operations and observe how aircraft requests and runway resources are handled.

24. Project Demonstration

During the final project demonstration, the following features will be shown:

- Launching the application.
- Main Menu.
- Runway Management.
- Aircraft Management.
- Aircraft Registration.
- Landing Request.
- Takeoff Request.
- Runway Assignment.
- Runway Occupancy.
- Emergency Handling.
- Priority Assignment.
- Runway Status Display.

- Aircraft Information Display.
- Operation Report.
- Summary Page.
- GitHub Repository.
- Opening the project in NetBeans.
- Running the complete application.

25. Final Release

The final version of the Airport Runway Management Simulator is prepared as Version 1.0. The final project repository can contain the following materials:

- Final Source Code
- Project Documentation
- Software Design Document
- Class Diagram
- Workflow Diagram
- User Manual
- Screenshots
- README File
- Final Project Files

The project can be downloaded from GitHub, opened in NetBeans, and executed for demonstration and further development.